Certificación de Entrega — Entrega 3: Comunicación síncrona y asíncrona

Módulo 3 — Comunicación y sincronización en sistemas distribuidos

> Qué evalúa esta entrega: que el grupo entienda cuándo corresponde cada semántica de comunicación, no solo que el sistema "funcione".

Datos generales

Campo	Valor
Grupo N°	*Completar* (Godoy – Iriarte – Lagoria)
Tag / Release de esta entrega	`v3-comunicacion`
Fecha de entrega	__ /__ / 2026

Checklist de la entrega

Requisito	Cumple	Justificación breve / evidencia
Hay un canal de comunicación síncrono funcionando entre componentes (ej. REST/gRPC).	**Sí**	`cliente-api` → `ia-inferencia` por HTTP (`POST /detectar`): manda un lote de teselas y espera las cajas detectadas. Probado contra el contenedor: `200` con las detecciones de cada tesela. Ver informe §2.
Hay un canal de comunicación asíncrono funcionando entre componentes (ej. cola de mensajes).	**Sí**	Cola persistente de entrenamientos: `ia-entrenamiento-api` encola y responde `202` en ~0,02 s, y los workers (contenedores aparte, escalables) consumen. Probado con 3 workers y 6 trabajos. Ver informe §3.
Está justificado por qué se usó cada semántica en cada punto del sistema.	**Sí**	Inferencia síncrona: el análisis necesita las cajas para seguir, la operación dura segundos y el servicio no tiene estado. Entrenamiento asíncrono: tarda horas, desacopla a la API de los workers y permite escalarlos. Ver informe §2 y §3.
Se identificó un caso de concurrencia concreto (ej. dos escrituras simultáneas sobre el mismo recurso).	**Sí**	Con varios workers, dos o más tomaban el mismo entrenamiento porque "leer el pendiente" y "marcarlo ejecutando" eran dos pasos separados. Además había lecturas de JSON a medio escribir. Ver informe §4.
Ese caso de concurrencia está resuelto y la solución puede demostrarse (no solo descripta).	**Sí**	Reclamo atómico con `.lock` (`O_EXCL`) y escritura atómica (temporal + `os.replace`). Script reproducible: código original con 55 ejecuciones para 20 trabajos (17 repetidos), con el arreglo exactamente 20 (0 repetidos).

Evidencia adicional

Cómo reproducir el caso de concurrencia resuelto. Desde la raíz del repositorio, sin Docker:

```
python docs/pdc/entrega-3-comunicacion/demo_concurrencia_cola.py --modo sin-arreglo   # muestra
trabajos repetidos
python docs/pdc/entrega-3-comunicacion/demo_concurrencia_cola.py --modo con-arreglo   # cada
trabajo, una sola vez
```

Con contenedores reales:

```
docker compose -f despliegue/compose.ia.yml up -d --build --scale ia-entrenamiento-worker=3
curl -X POST localhost:8002/entrenamientos -H "Content-Type: application/json" -d '{"datasets":
["demo"],"epocas":1}'
curl localhost:8002/entrenamientos    # cada trabajo muestra el único worker que lo tomó
```

- Script: `demo_concurrencia_cola.py`
- Salidas: `salida-codigo-original.txt` · `salida-con-arreglo.txt` · `salida-docker-3-workers.txt`
- Código del arreglo: `project/servicios/ia-entrenamiento/nucleo/trabajos.py` y `worker/entrenador.py`
- Informe: `informe.md`

Espacio para el docente

Nota	Corregido por	Fecha de corrección
___ / 10		__ /__ / ___

Observaciones: